

Javascript

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Hoisting</title>
  </head>
  <body>
    <h2>Hoisting</h2>

    <script>
      // console.log(data);
      // var data = 10; //compilation phase , it's create memory

      // // browser => javascript engine
      // // chrome => v8

      // console.log(hoist);

      // let hoist = 30;

      // display();

      // function display(){
      //   console.log("display function");

      // }

      // var data ;
      // let hoist;
      // function display(){
      //   console.log("display function ");

      // }

      // console.log(data);

      // var data = 10;

      // console.log(hoist);

      // let hoist = 10 ;

      // display();
```

```
// function display(){
//   console.log("display function");
// }

// func();
// console.log(func());

// try {
//   arrow();
// } catch (error) {
//   console.log(error.message);
// } finally {
//   console.log("finally block ");
// }

// let age = 10;

// if (age >= 18) {
//   console.log("You are eligible to vote");
// } else {
//   throw new Error("You are eligible to vote ");
// }

// var func = function fiit() {
//   console.log("fiit");
// };

// let arrow = () => {
//   console.log("arrow function ");
// };

// console.log("End of the program ");

// callback

function greet(name, callback) {
  console.log(name);
  callback();
}

function sayhello() {
  console.log("Welcome");
}
greet("Sancia", sayhello);

// closure
```

```
function outer() {
  let count = 0;

  return function inner() {
    count++; // lexical scope (closure)
    console.log(count);
  };
}

let result1 = outer();

result1();

function add(a, b) {
  return a + b;
}

let result = add(10, 20);

console.log(result);

class Student {
  static count = 1;
}

let obj = new Student();

obj.count++;
// console.log(obj.count);
Student.count++;
Student.count++;
console.log(Student.count);
console.log(Student.count);

let obj2 = new Student();

obj2.count++;

// console.log(obj2.count);

// window events

window.onload = function () {
  console.log("webpage loaded successfully");
};

window.onresize = function () {
  console.log("Resized");
};
```

```

    };

    window.onscroll = function () {
        console.log("Scrolled");
    };

    console.log(window.innerHeight);

    console.log(window.innerWidth);

    console.log(screen.width);
    console.log(screen.height);

    console.log(3 & 4);

</script>
</body>
</html>

```

Interview-Based Theory

1. Hoisting

Definition: Hoisting is JavaScript's default behavior of moving declarations to the top of the scope during compilation. **Key points:**

- var → hoisted but initialized as undefined.
- let/const → hoisted but remain in the **Temporal Dead Zone (TDZ)** until initialized.
- Function declarations → fully hoisted.
-

Syntax:

```
console.log(a); // undefined
```

```
var a = 10;
```

```
hoist(); // works
```

```
function hoist(){ console.log("Hoisted function"); }
```

2. Callback Functions

Definition: A callback is a function passed as an argument to another function. **Use case:** Asynchronous programming, event handling.

Syntax:

```
function greet(name, callback){  
  
  console.log(name);  
  
  callback();  
  
}
```

3. Closures

Definition: A closure is a function that remembers variables from its lexical scope even after the outer function has finished executing. **Use case:** Data privacy, maintaining state.

Syntax:

```
function outer(){  
  
  let count = 0;  
  
  return function inner(){ count++; console.log(count); };  
  
}
```

4. Classes and Static Properties

Definition: ES6 classes provide a blueprint for creating objects. **Key point:** static properties belong to the class, not instances.

Syntax:

```
class Student {  
  
  static count = 0;  
  
}
```

Student.count++; // modifies class property

5. Window Events

Definition: Events triggered by browser actions. **Examples:**

- window.onload → fires when page loads.
- window.onresize → fires when window is resized.
- window.onscroll → fires when page is scrolled. **Syntax:**

```
window.onload = () => console.log("Page loaded");
```

Coding Round Questions

- Hoisting → Explain hoisting with var, let, and const.
- Callback → Write a callback function that prints a greeting.
- Closure → Write a closure that maintains a counter.
- Static property → Demonstrate static property usage in a class.
- Window events → Write code to log messages on load, resize, and scroll.

⚡ Machine Coding Round Challenge

Problem: Simple Voting System

Requirements:

- Create a Voter class with static property totalVotes.
- Each voter object has name and age.
- Use a function vote() that checks eligibility (age >= 18).
- If eligible, increment totalVotes.
- Use **closure** to maintain a private counter of invalid votes.
- Use **window.onload** to display results when page loads.